Data Structures

Comp Sci 214, Spring 2025

Bad programmers worry about the code.

Good programmers worry about data structures and their relationship.

— Linus Torvalds

Show me your flowcharts [functions] and conceal your tables [data structures], and I shall continue to be mystified.

Show me your tables [data structures], and I won't usually need your flowcharts [functions]; they'll be obvious.

— Fred Brooks

Data structures are fundamental when writing any but the most trivial of programs.

So, what is a data structure?

A scheme for organizing data to use it effectively and efficiently.

Two parts

- **Representation**: how to map the conceptual pieces of your data (e.g., a person's name, or their phone number) to concrete building blocks (e.g., structs, vectors, etc.)
- **Operations**: how can a program access and manipulate these conceptual pieces to do its work

Goals of any data structure

A scheme for organizing data to use it *effectively* and *efficiently*.

Effectively: correctness

- Operations have the behavior that they promised
- **Why?** So our programs can rely on them!

Efficiently

- **Time**: operations should be fast
- **Space**: representations should not use too much memory
- Locality, power usage, etc.
- **Why?** Results sooner, fewer servers, fewer emissions, etc.

An **abstract data type** defines a class of abstract objects which is *completely* characterized by the *operations* available on those objects.

— Barbara Liskov

Abstract Data Types (ADTs)

Data structures are all about tradeoffs!

- Often impossible to meet all their goals 100%
- Different data structures pick different tradeoffs
- No perfect data structure that is always the best!

ADT: *group* of data structures that can serve the same role.

- Each with its own representation and operations.
- Each meets different goals to different degrees.
- You can choose one or another based on which *tradeoffs* are best for **your** situation.
- Can *switch* between them as your needs change!

One of our guiding notions in this class.

We'll have a full lecture on those, but here's a teaser.

Example: Set ADT

Let's say we want to use sets of integers in our program.

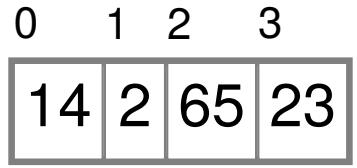
- We want to be able to insert new integers into sets
- And check whether a given integer is in the set

(Real sets have more operations; these will do for today.)

"Set" is an Abstract Data Type

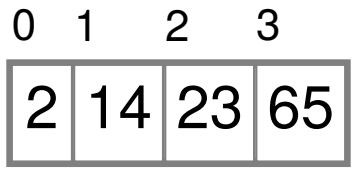
- Multiple different data structures could work in our program!
- Here are three possibilities:
  - Vector (array)
  - *Sorted* vector
  - Linked list
- Let's represent the set $\{14, 2, 65, 23\}$ with each one

Example: Vector-based set



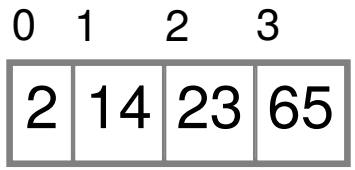
- **Representation**: set elements = vector elements
- **Operations**:
  - **Insertion**: Create new, bigger vector. Copy elements over.
    - **True** vectors cannot grow!
    - If what you're using can grow, then it's a different DS!
  - **Membership**: Loop through vector, check each element.

Example: Sorted-vector-based set



- **Representation**: *sorted* set elements = vector elements
  - I.e., elements are stored in increasing order
- **Operations**:
  - **Insertion**: Create new, bigger vector. Copy elements over, inserting new one in the right place
  - **Membership**: Binary search!
    - If you don't know what that is, it's ok! We'll see it later.
    - More efficient than looping through vector!

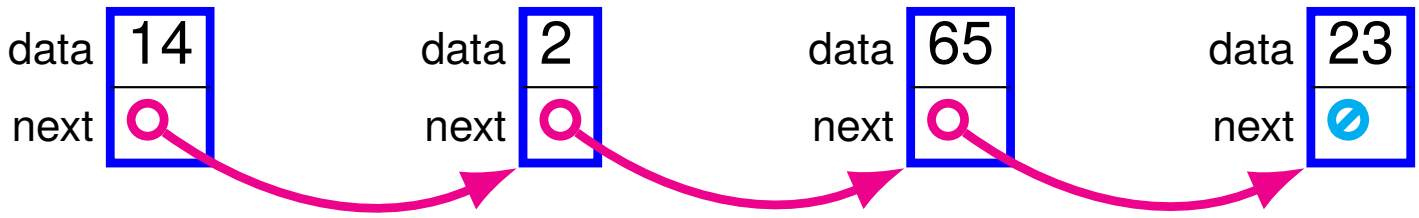
Example: Sorted-vector-based set



- **Representation**: *sorted* set elements = vector elements
  - I.e., elements are stored in increasing order
- **Operations**:
  - **Insertion**: Create new, bigger vector. Copy elements over, inserting new one in the right place
  - **Membership**: Binary search!
    - If you don't know what that is, it's ok! We'll see it later.
    - More efficient than looping through vector!

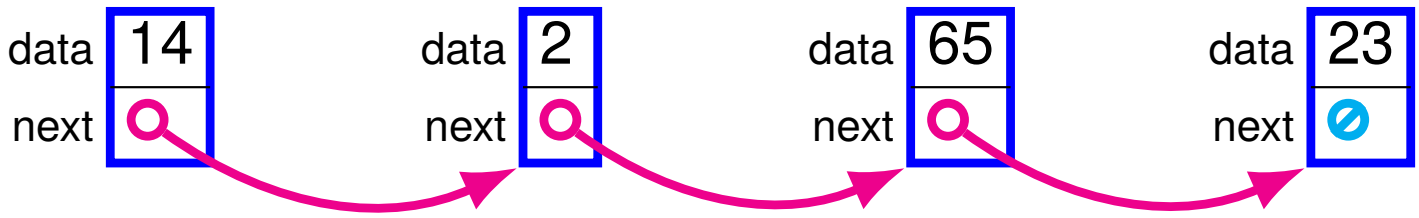
- This representation has an *invariant*
  - The contents of the array **must** be sorted
  - Otherwise, operations will go wrong!
  - We'll talk a lot of more about invariants this quarter

Example: Linked-list-based set



- **Representation**: set elements = linked list elements
  - We will talk more about linked lists next time.
- **Operations**:
  - **Insertion**: Add a new node to the linked list. Instant!
  - **Membership**: Loop through the list, check each element.

Example: Linked-list-based set



- **Representation**: set elements = linked list elements
  - We will talk more about linked lists next time.
- **Operations**:
  - **Insertion**: Add a new node to the linked list. Instant!
  - **Membership**: Loop through the list, check each element.

- Three data structures, all implementing the set ADT!
- Different tradeoffs!
  - If we only ever check membership, sorted array is best
  - If we mostly insert new elements, linked list is best

Zooming Out

Goals of this class

See a wide range of data structures

- So you have a *large vocabulary* when designing programs
- So you can make *informed choices* of ADTs and DSs

Goals of this class

See a wide range of data structures

- So you have a *large vocabulary* when designing programs
- So you can make *informed choices* of ADTs and DSs

Zoom out from programming to engineering

- *Larger building blocks*: DSs, ADTs, algorithms
  - Contrast 111/150/211: smaller scale with functions, conditionals, loops, etc.
- *Bigger picture concerns*: interfaces, testing, design, etc.
  - I.e., writing programs that *others can build on and rely on*

Goals of this class

See a wide range of data structures

- So you have a *large vocabulary* when designing programs
- So you can make *informed choices* of ADTs and DSs

Zoom out from programming to engineering

- *Larger building blocks*: DSs, ADTs, algorithms
  - Contrast 111/150/211: smaller scale with functions, conditionals, loops, etc.
- *Bigger picture concerns*: interfaces, testing, design, etc.
  - I.e., writing programs that *others can build on and rely on*

Solve problems like a computer scientist

- *There is more to computer science than programming!*
- Conceptual and theoretical foundations
  - Complexity, algorithms, invariants, correctness, etc.
  - Allows us to tackle *more complex and interesting problems*!
- **Caution**: don't neglect this aspect of the class!
  - You'll miss a lot! And be ill-prepared for future classes.

Course philosophy

Learning comes first, everything derives from that

- Last stop before 300-level classes / internships
  - You need to get ready for them!

Course philosophy

Learning comes first, everything derives from that

- Last stop before 300-level classes / internships
  - You need to get ready for them!

Low stakes

- Room to learn from your mistakes (wherever possible)
  - E.g., feedback and resubmissions for programming hws
- Bounded impact (otherwise)
  - I.e., you won't fail the class just by bombing one exam
- Flexibility for everyone, built-in
  - No questions asked, no need to ask
  - Late tokens, redo tokens, etc.; see syllabus for details

High expectations

- We expect you to *consistently* produce *excellent* work
- One doesn't learn much by producing mediocre work
  - So that won't be enough to pass the class

Course philosophy

Resubmissions, not partial credit

- In other classes: half-broken work $\rightarrow$ partial credit
  - But you missed out on half the learning!
  - Moving on is a big missed opportunity!
- In this class: half-broken work $\rightarrow$ no credit, **go fix it!**
  - You still have concepts / good habits to learn
  - And you will get credit once you learn them!

Transparency

- Grading mechanisms, expectations, etc. stated **up front**
  - Everything is in the syllabus and in assignment handouts
- If you're headed for trouble, you can see it coming
  - And I'll reach out in case you need help

You're in control

- Your grade is determined entirely by your own work
- **Not** by the work of of your classmates
  - No curves, percentage cutoffs, etc.

Course philosophy

I'm confident all of you can succeed

- But you'll have to work for it, and be diligent!
- Course staff and I here to help!
  - Reach out if you need help (see syllabus)
  - I'll also reach out if you look like you're in trouble

How to succeed in this class

- **Today**: read the syllabus, in full
- **Before lectures**: read textbook chapter, skim slides
  - So you have an idea of what we'll be talking about
  - And so you're ready to ask questions!
- **During lectures**: ask questions!
  - Either by raising your hand, or anonymously on Piazza
  - If you can't make it to lecture, watch the recording ASAP
- **After lectures**: review what we have seen
  - If something is not clear, ask on Piazza
- **Start assignments early**: they'll get big
  - First one *out now*! Due next week

Classroom etiquette

We take attendance, and we expect you to be here

- This class is for you. If you're not here, you don't benefit!
- **I have seen**: not coming to class $\rightarrow$ very high risk of failing
  - I don't want this to happen to you!
- 50% attendance required to pass; no effect otherwise
  - Not a high bar; but we expect more than that
  - If you make a habit of coming to class, you'll be fine
  - If you make a habit of skipping, you'll be in danger
  - Full details in syllabus

- We'll be using the YouHere app to track attendance
  - Info in syllabus
  - Check in with the app at the **start** of lectures, from this room
  - Test it out for lecture 2, we start counting with lecture 3

- **Keep in mind**: this is a guardrail, not a punishment!
  - If you're falling behind; I'll reach out

Classroom etiquette

- Learning this material requires focus and concentration
- Let's make our classroom environment conducive to that
  - **Goal**: minimize distractions and disruptions

- Laptops in class are fine
  - But if you do non-class-related things, please sit in the back
  - So you don't distract your colleagues behind you
- One conversation at a time, please
  - If someone is talking (myself, another student), don't have another conversation on the side
    - Distracting to myself and to your colleagues
  - You have a question? Awesome!
    - Raise your hand or ask on Piazza; don't ask your neighbor
    - That way everyone benefits!
- Coming in late / leaving early?
  - Please sit in the back, and come in / leave quietly

Office hours etiquette

We have a great team who is here to *help you learn*.

Some advice to make the most of office hours:

- **Be prepared**: be ready to show us what you've tried.
  - If you're not prepared, **our staff will not help you**.
- **Be specific**: we can't read your mind.
  - **You** know what you're struggling with better than we do.
  - If you're precise, we can help you much more effectively.
- **Be proactive**: we are not here to fix your bugs for you.
  - We are here to **get you to the point** where you can **solve your problems yourself**.
  - Doing it for you would be **doing you a disservice**

This class is not about getting data structures to work.
It's about getting *you* to be a competent computer scientist.

Academic integrity

We take this very seriously, and so should you.

The work you submit must be entirely your own

- No unauthorized sources
  - This includes peers, online sources, *any* AI tools, etc.
- No excessive collaboration
- Full details in syllabus; stricter than many classes!

Action item: print and sign the class's policy, in the syllabus

- Bring signed copy next class

Academic integrity

We take this very seriously, and so should you.

The work you submit must be entirely your own

- No unauthorized sources
  - This includes peers, online sources, *any* AI tools, etc.
- No excessive collaboration
- Full details in syllabus; stricter than many classes!

Action item: print and sign the class's policy, in the syllabus

- Bring signed copy next class

Consequences (you don't want any of that!)

- **Biggest one**: you won't learn!
  - This material is *foundational*; need it for everything else!
  - Can't do it without a crutch? $\rightarrow$ you'll be cooked later on!
- **Report to dean**: No credit, Fs, suspensions, expulsions
  - **Warning**: Can't drop the class after misconduct!

Academic integrity

We take this very seriously, and so should you.

The work you submit must be entirely your own

- No unauthorized sources
  - This includes peers, online sources, *any* AI tools, etc.
- No excessive collaboration
- Full details in syllabus; stricter than many classes!

Action item: print and sign the class's policy, in the syllabus

- Bring signed copy next class

If you're struggling, ask us for help!

- We're here to help!
- Lots of resources at your disposal

If it looks like you're struggling; I'll reach out.

- I much prefer spending an hour helping you succeed…
- …than spend that same hour reporting you to the dean.

Course staff

Instructor	Vincent St-Amour
Teaching assistant	Tochukwu Eze
Peer mentors	Adedamola Adejumobi, Alison Bai, Nico Bers, Caroline Guerra, Trenton Kim, Jason Latz, Miya Liu, Kaylie Mei, Omotayo Oludemi, Ethan Piñeda, Shreya Sridhar, and Burke Stanton

Our language: DSSL2

Our language: DSSL2

DSSL2: Data Structures Student Language (version 2)

Close cousin of Python

- If you know Python, learning DSSL2 should be easy
- If you know DSSL2, learning Python should be easy

Our language: DSSL2

DSSL2: Data Structures Student Language (version 2)

Close cousin of Python

- If you know Python, learning DSSL2 should be easy
- If you know DSSL2, learning Python should be easy

Why not Python itself?

- Has tons of useful data structures already!
  - One big reason many programmers like it
  - But not appropriate for a data structures class
- But doesn't have the building blocks we need!
  - E.g., no basic arrays (but something better instead!)

The concepts we will learn are language-independent anyway!

Installing DSSL2

- DSSL2 is built on top of Racket
  - But DSSL2 $\neq$ Racket! Very different languages.
- Install Racket 8.16 (latest version) from `racket-lang.org`
  - Older versions will not work properly later on
- In DrRacket, File $\rightarrow$ Package Manager
  - Install with source `dssl2`
- Don't want to use DrRacket when programming?
  - No problem! See syllabus for alternatives.

DSSL2 expressions and statements

```
3 + 5              # comments start with #

6 * (3 + 5)

1 + 'hello'.len()  # method call


let x = 5          # slight difference from python
                   # to avoid ambiguity (-> bugs)


if condition:      # indentation matters! just like python
    do_some_stuff()
else:
    do_other_stuff(x, y, z)  # function call
```

DSSL2 functions and programs

```
#lang dssl2
# Every DSSL2 program starts with this `#lang` line.

# hypotenuse(Number, Number) -> Number
# Finds the length of the hypotenuse given the lengths
# of the other two sides of a right triangle.
def hypotenuse(a, b):
    if (a < 0 or b < 0):
        error('cannot have negative-length sides!')
    return (a * a + b * b).sqrt()

# ack(Natural, Natural) -> Natural
# Computes Ackermann numbers.
# Those will do a guest appearance later.
def ack(m, n):
    if   m == 0: return n + 1
    elif n == 0: return ack(m - 1, 1)
    else:        return ack(m - 1, ack(m, n - 1))
```

DSSL2 assertions and test cases

Programming without tests is like running around with a blindfold and a blowtorch.

— Prem Seetharaman

```
# assertions error and stop your program if they fail
# great for error checking and testing!
assert ack(0, 4) == 5

test 'arithmetic works':     # can give a name to tests
    assert 2 * 5 == 10       # can group assertions
    assert 2 + 5 == -1       # errors, then continues
                             # running after test block
    assert 2 - 5 == -3
```

In this class, we expect you to write your own tests.

See supplementary video and document on Canvas for advice.

What are data structures made of?

Ultimately: bits and bytes

- Like anything else on a computer
- But we won't worry about that level; that's Comp Sci 213

Conceptually: *boxes* and *arrows*

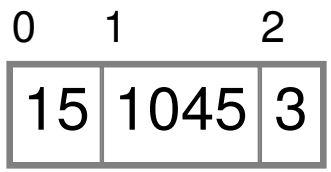
- Boxes hold information
- Arrows connect boxes to create larger structures

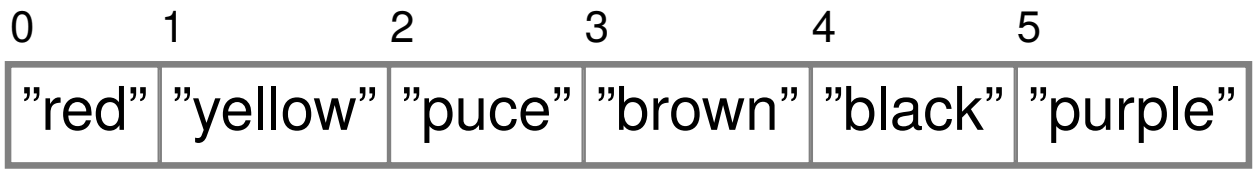
Two kinds of boxes

- *Vectors* (or *arrays*)
- *Structs*

Vectors

- Vectors contain sequences of data
  - *Indexed* by *integers* $0 \ldots n-1$
  - Contents typically all of the same type
- Size fixed when *creating* the *specific vector*
  - A vector is created with size 10 $\rightarrow$ it has size 10 forever
  - But we can create different vectors of different sizes
- Vectors take the same time to access an element…
  - Regardless of how far into the vector!
    - Element #1? Element #1,000,017? Same cost!
  - Regardless of how big the vector is!
    - 10-element vector? 10,000,200-element vector? Same cost!





Vectors, in DSSL2

```
# Can construct vector literals.
let v = [ 0, 1, 1, 2, 4, 7, 13, 24, 44, 82 ]

test 'vector basics':
    assert v[3] == 2
    assert v[6] == 13
    assert v.len() == 10

test 'vector set':
    v[6] = 23
    assert v[6] == 23
```

Vectors, in DSSL2

```
# Can construct vector literals.
let v = [ 0, 1, 1, 2, 4, 7, 13, 24, 44, 82 ]

test 'vector basics':
    assert v[3] == 2
    assert v[6] == 13
    assert v.len() == 10

test 'vector set':
    v[6] = 23
    assert v[6] == 23

# Vector comprehensions allow you to create a vector
# using a "description" rather than literal elements.
[ 0; 1000000 ]
# Creates a vector with `1000000` elements, all `0`s.
# Much nicer than typing the whole thing!
# Supports more complicated descriptions too, see docs.
```

Looping over vectors

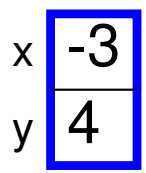
```
# sum(Vector<Number>) -> Number
# Sums the elements of a vector.
def sum(vec):
    let result = 0
    # for-each loop, like in python or C++
    # `v` holds each element of the vector, in turn
    # first iteration,  `v` is `vec[0]`
    # second iteration, `v` is `vec[1]`
    # etc.
    for v in vec:
        result = result + v
    return result

# Can also loop over indices with `range`, but less nice.
```

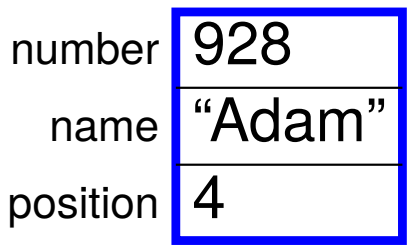
Structs

- Structs contain collections of data
  - *Accessed* by field *names*
  - Fields can be (and often are) of different types
- Fields determined when *defining* the struct *type*
- Accessing any field takes the same amount of time
  - Again regardless of which or how many fields

2D positions have an `x` and a `y` field, both numbers.

Runners have a `number` field (integer), a `name` field (string), and a `position` field (integer).



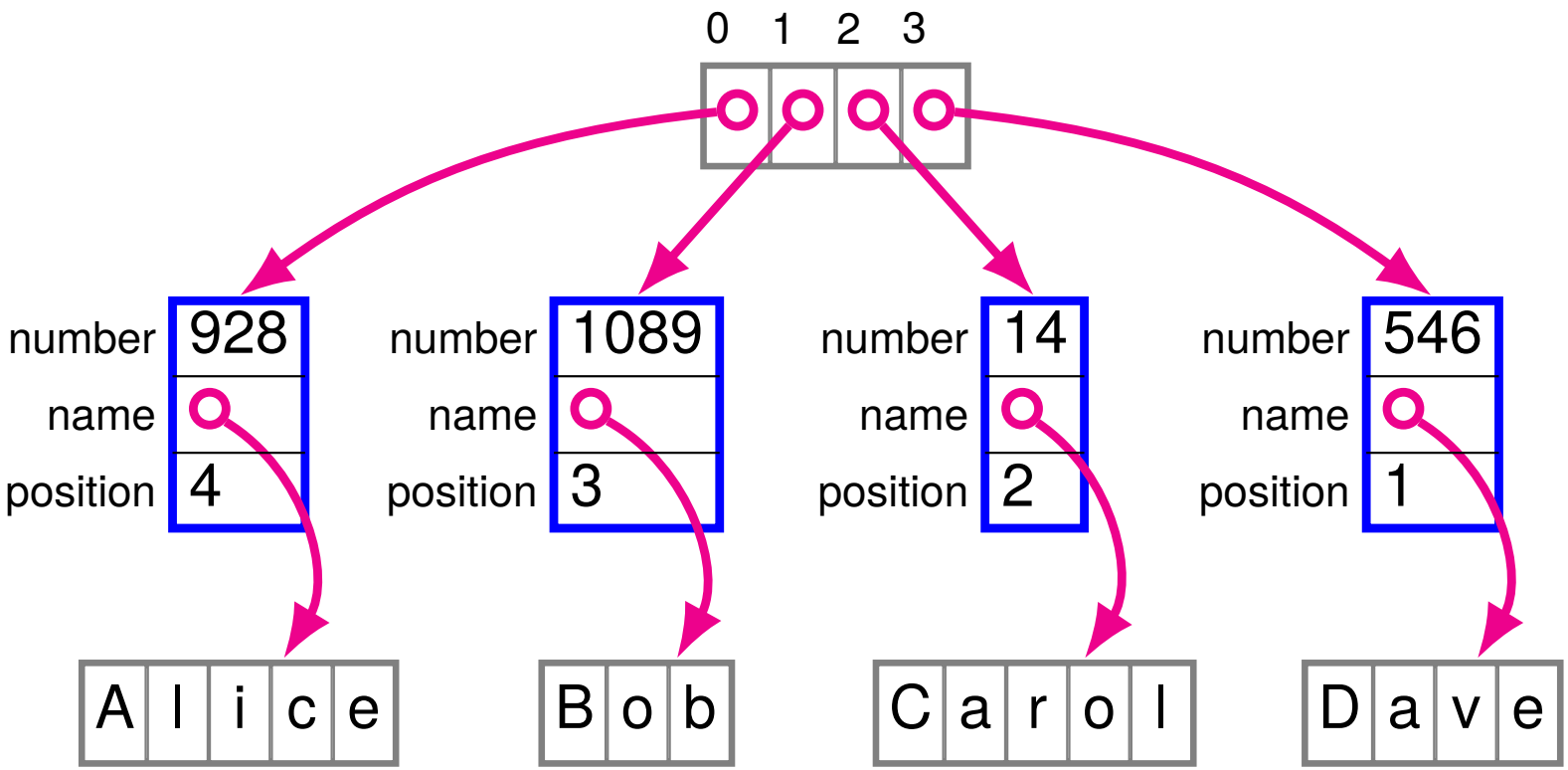


Structs, in DSSL2

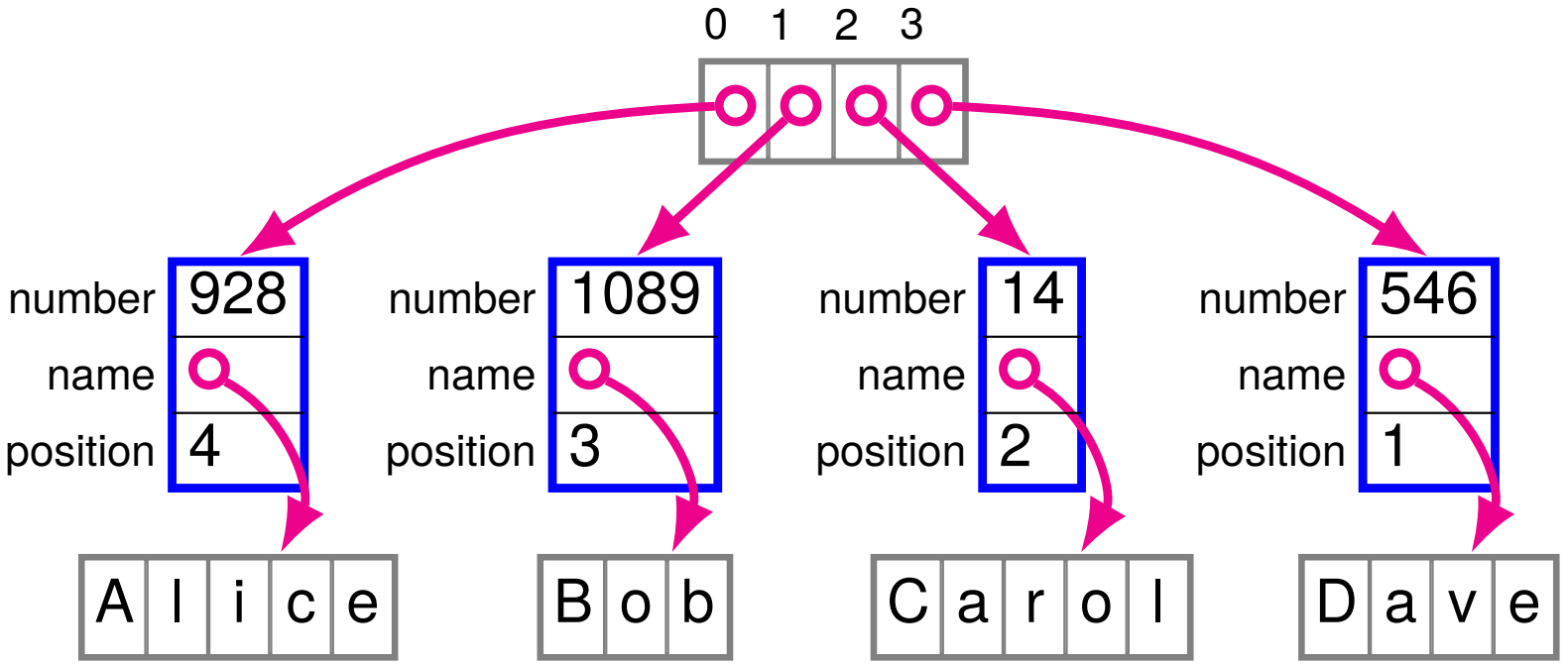
```
# Defines a new type of structure `posn` with fields
# `x` and `y`:
struct posn:
    let x
    let y

let p = posn(3, 4)

assert posn?(p)      # asserts that the result is true
assert p.x == 3
assert p.y == 4      # uses `.` notation like Python, etc.

p.x = 6
assert p.x == 6
assert p.y == 4
```

Arrows

- Arrows allow boxes to refer to one another
  - And thus to build arbitrarily-sized data
- *Implicit* in DSSL2 (and Python, etc.)
  - Struct in a struct field = actually an arrow to that struct!
  - Contrast pointers and references in C++ (*explicit* arrows)



Arrows, in DSSL2




```
struct runner:
    let number; let name; let position

let runners = [ runner( 928, "Alice", 4),
                runner(1089, "Bob",   3),
                runner(  14, "Carol", 2),
                runner( 546, "Dave",  1) ]
```

Working with structs and vectors

```
struct runner:
    let number; let name; let position

let runners = [ runner( 928, "Alice", 4),
                runner(1089, "Bob",   3),
                runner(  14, "Carol", 2),
                runner( 546, "Dave",  1) ]
```

QUIZ: Suppose we want to find out Carol's position:

A: `runners[3].position`

B: `runners[2].position`

C: `runners.position[2]`

D: `runners->position[3]`

Working with structs and vectors

```
struct runner:
    let number; let name; let position

let runners = [ runner( 928, "Alice", 4),
                runner(1089, "Bob",   3),
                runner(  14, "Carol", 2),
                runner( 546, "Dave",  1) ]
```

QUIZ: Suppose we want to find out Carol's position:

A: `runners[3].position`

B: `runners[2].position`

C: `runners.position[2]`

D: `runners->position[3]`

Working with structs and vectors

```
struct runner:
    let number; let name; let position

let runners = [ runner( 928, "Alice", 4),
                runner(1089, "Bob",   3),
                runner(  14, "Carol", 2),
                runner( 546, "Dave",  1) ]
```

QUIZ: Carol passed Dave in the race, how do we reflect that? (I.e., Carol's `position` goes from 2 to 1)

A: `runners[1].position = 2`

B: `runners.position[2] = 1`

C: `runners[2].position = 1`

D: `runners[2].position = 2`

Working with structs and vectors

```
struct runner:
    let number; let name; let position

let runners = [ runner( 928, "Alice", 4),
                runner(1089, "Bob",   3),
                runner(  14, "Carol", 2),
                runner( 546, "Dave",  1) ]
```

QUIZ: Carol passed Dave in the race, how do we reflect that? (I.e., Carol's `position` goes from 2 to 1)

A: `runners[1].position = 2`

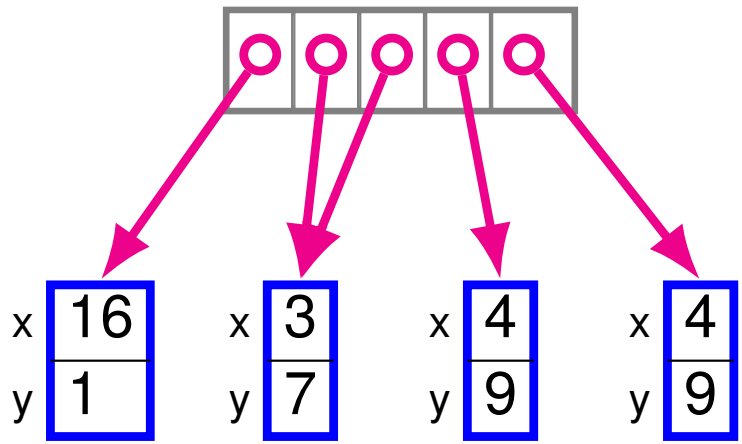
B: `runners.position[2] = 1`

C: `runners[2].position = 1`

D: `runners[2].position = 2`

Box identity

- Boxes have *identity*: each box is its own object
  - Two boxes with same contents may not be the same box!
- Boxes can be *aliased*:
  - Two arrows can point to the same box
  - A change through one arrow can be seen through the other!



Heads up: if you see output like `#0#` in DSSL2, that's aliasing. Cf. supp. video on DSSL2 error messages (on Canvas)

Vectors vs structs

When should you use one versus the other?

- Do you have a **fixed** number of fields **you can name**?
  - Use a **struct**

- Does the number of elements **depend** on external factors?
  - Use an **array**
  - Bonus hint: Elements are all of the same type

Classes

- Structs and vectors are enough to **represent** any data
- But data structures = representation + **operations**
  - Classes allow us to combine the two
- Classes ≈ structs + functions (methods)
  - A code organization mechanism to group representation and operations together

Classes, in DSSL2

```
class Posn:
    let x                           # fields: initialized by
    let y                           # the constructor

    def __init__(self, x, y):      # constructor: method
        self.x = x                  # with a special name
        self.y = y

    def get_x(self): self.x        # `return` is optional
    def get_y(self): self.y        # `self` = receiver

    def distance(self, other): # some other method
        # fields are private to individual objects
        # need to use getter for `other`
        let dx = self.x - other.get_x()
        let dy = self.y - other.get_y()
        return (dx * dx + dy * dy).sqrt()
```

Classes, in DSSL2

```
let p = Posn(3, 4)
assert p.get_x() == 3
assert p.get_y() == 4
assert_error p.x                # fields are private
                                # cannot access them directly
                                # from outside

let q = Posn(0, 0)
assert p.distance(q) == 5
```

Contracts

DSSL2 (like Python) does not have static types.

Instead, you can use contracts to check values.

- And catch mistakes before they snowball!

```
def add_elt (x: int?, i: nat?, v: VecC[int?]) -> int?:
    return x + v[i]

# contract violation! expected `nat?` (i.e., int >= 0)
assert_error add_elt(10, -5, [2, 3, 4])
```

You can use contracts if you want, but you don't have to.

We'll provide some contracts to help you in assignments.

See supplementary video on Canvas (and docs) for details.

For more DSSL2 information

See the DSSL2 reference (or help desk).

To search the help desk for DSSL2-specific topics
(instead of every package under the sun):

- Prefix your query with `T:dssl2`
- For example, `T:dssl2 error` to search for DSSL2's `error` function

Larger example: `recipes.rkt` on Canvas.

See also supplementary videos on Canvas.

Homework 1 is there to help you get used to the language.

We will start next lecture with some time for DSSL2 Q&A.

Next time: Linked lists